Week1

什么是software

这里PPT上给出的是wikipedia上的定义

Computer software, or simply software, is a collection of data or computer instructions that tell the computer how to work. This is in contrast to physical hardware, from which the system is built and actually performs the work.

计算机软件，或简称软件，是一组数据或计算机指令，告诉计算机如何工作。这与物理硬件形成对比，物理硬件是构建系统并实际执行工作的基础。

In computer science and software engineering, computer software is all information processed by computer systems, programs and data.

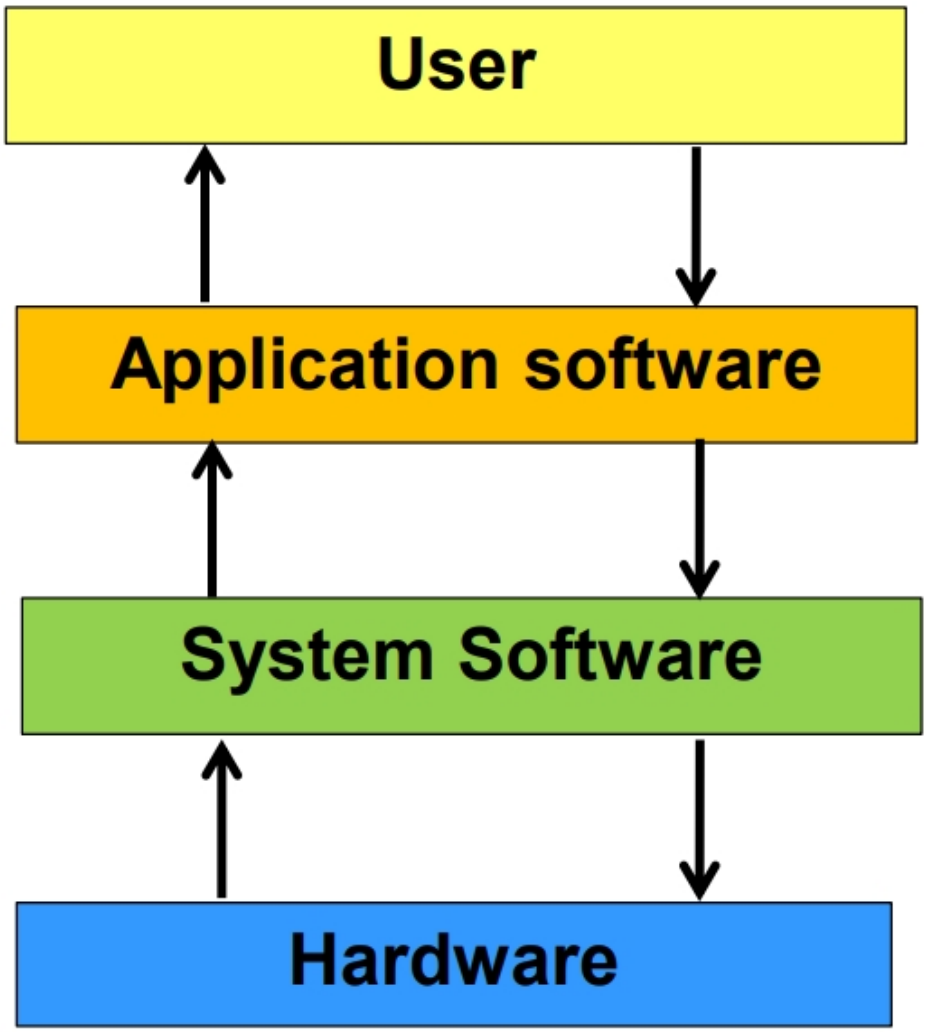
在计算机科学和软件工程中，计算机软件是由计算机系统、程序和数据处理的所有信息。

Software types

分为两种

- System software 系统软件
    - Operating systems 操作系统
    - Device drivers 设备驱动程序
    - Utilities 实用软件
- Application software (Perform a specific task) 应用软件(具体任务执行)
    - Word processing 文字处理
    - Image processing 图像处理
    - Social network 社交网络

两种软件与User和Hardware之间的关系



Generic Software vs Custom Software

- Generic Software 通用软件
  - Developed for a general market 为一般市场开发的
  - To be sold to a range of different customers 卖给一系列不同的客户
  - Example: Microsoft Office, Photoshop
  - Owned and controlled by the development organisation 由开发组织拥有和控制的
- Custom software 定制软件
  - Developed for a particular customer, according to their specific needs 针对特定的客户，根据他们的特定需求开发
  - Examples: software used in banks, airlines, embedded systems

    - Owned and controlled by the customer organisation 由客户组织拥有和控制

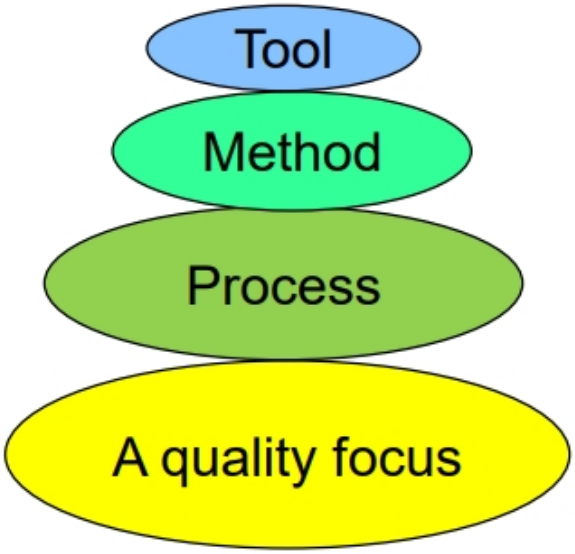
- 现在的趋势是Generic to Custom
    - More and more, software companies are starting with a generic system and customising it to the needs of a particular customer. 越来越多的软件公司从通用系统开始，并根据特定客户的需求对其进行定制。

The feature of Good Software

- Delivers required functionality 提供所需的功能
- Usable 可用的
    - Usable by the users (or systems) it is designed to interact with. 可供设计与之交互的用户(或系统)使用。
- Efficient 高效的
    - Good use of resources: computational, user time, development time/cost 充分利用资源:计算、用户时间、开发时间/成本
- Dependable 可靠的
    - Robust, reliable, trustworthy 健壮，可靠，值得信赖
- Maintainable 可维护的
    - can evolve to meeting changes in requirements. 可以发展以满足需求的变化
- Understandable 可理解的
- Cost-effective 划算的
- Secure/Safe 安全的

Software engineering layers

Software engineering is a layered technology —— 4 layers



【↓从下往上分别是】

- Commitment to quality 对质量的承诺
- Process: foundation layer 过程:基础层
- Methods: technical layer 方法:技术层
- Tools: support layer 工具:支持层

Software failures

Large scale software development failure

- Numerous examples of failed or seriously delayed software development projects 失败或严重延迟的软件开发项目的例子不胜枚举
- Or fail to deliver full functionality within time and budget. 或者未能在时间和预算内交付完整的功能。
- Or disasters: 或者灾难
    - E.g., NHS IT System, Heathrow T5, Pensions system,Taurus (Stock Exchange), Air Traffic Control, ESA Arianne 5, Patriot Missile System, Therac-25 radiation therapy machine

Small scale software development failure【这个应该是PPT省略了，反正就是说还有小规模的】

present reality

现状反正就是很复杂很mixed，然后software design还是difficult，这里就不详细写了

General issues that affect software

- Heterogeneity 异质性
    - distributed systems, across networks, different types of devices 分布式系统，跨网络，不同类型的设备
- Business and social change 商业和社会变革
    - emerging economies develop, new technologies 新兴经济体不断发展，新技术层出不穷
- Security and trust 安全性和信任
    - it is essential that we can trust that software 至关重要的是，我们可以信任软件
- Scale 规模
    - Software has to be developed across a very wide range of scales 软件必须在非常广泛的范围内开发

Software Process

A software process is a set of structured activities that leads to the production of the software

软件过程是引出软件生产的一组结构化活动

Many different software processes but all involve:

许多不同的软件过程，但都涉及:

- Requirement specification 要求规格说明
- Development 发展
    - Analysis 分析
    - Design 设计
    - Implementation 执行

- Validation (Testing/Deployment) 验证(测试/部署)
- Evolution (Maintenance) 进化/迭代（维护）

Requirement specification 要求规格说明

Defining what the system should do

定义系统应该做什么

Aim: a **complete description** of the problem and of the constraints imposed by/on the environment

目的:对问题和环境所施加的限制的完整描述

Description may contain: 描述可能包含:

- Functions of the system 系统的功能
- Future extensions 未来的扩展
- Amount of documentation required 所需文件数量
- Response time and performance 响应时间和性能
- Acceptance criteria 验收准则

Description的Result: Requirements specification

Development

Analysis

Aim: analyse requirements to create a conceptual model of the software system.

目的:分析需求，建立软件系统的概念模型。

- Data modelling 数据建模
- Functional modelling and information/control flow 功能建模和信息/控制流
- Behavioural modelling and state 行为模型和状态
- User interface modelling 用户界面建模

Analysis的Result: a set of Analysis Models

结果:一组分析模型

Design

Aim: an implementable model of the software system.

目的:建立一个可实现的软件系统模型。

- Sufficient information to allow system to be built. 建立系统所需的足够信息。

Architecture is defined.

架构是定义好的。

System is decomposed to components within the architecture.

系统被分解为体系结构中的组件。

Design decisions dramatically impact system quality.

设计决策极大地影响系统质量。

Design的Result: Detailed Design Documentation

结果:详细设计文档

Implementation

Aim: implementation of all design elements.

目的:实现所有设计元素。

Starts from the component specifications developed during design.

从设计过程中开发的组件规范开始。

- Interfaces defined in the design should be respected by the implementation of the component. 在设计中定义的接口应该得到组件实现的尊重。

Code should be well documented and easy to read, flexible, correct, reliable AND fully tested.

代码应该有良好的文档，易于阅读，灵活，正确，可靠，并经过充分测试。

Implementation的Result: working software

Testing

Checking that it does what the customer wants;

- Unit testing 单元测试
- Functional testing 功能测试
- Integration testing 集成测试

- System testing 系统试验
- Acceptance testing 验收测试
- Testing and implementation should run in parallel 测试和实现应该并行运行

Testing的Result: Fully tested software

Deployment 部署

Implement a strategy to avoid outages

实施避免中断的策略

Package software ready to install on a computer system/device or deploy to servers

可以安装在计算机系统/设备上或部署到服务器上的软件包

Actually installing/ deploying the software.

实际安装/部署软件。

Live testing in real environment.

在真实环境中进行现场测试

Documentation and manuals

文档和手册

Training

训练

Deployment的Result: Working software in real environment

Evolution

Aim: keeping the system operational after delivery to the customer; changing the system in response to changing customer needs.

目的:在交付给客户后保持系统的可操作性;改变系统以响应不断变化的客户需求。

- Corrective: identification and removal of faults. 纠正:故障的识别和排除。
- Adaptive: modifications needed to achieve interoperability with other systems and platforms. 自适应:需要修改以实现与其他系统和平台的互操作性。
- Perfective: incorporation of new features, improved performance and other modifications. 完善的:加入新的特性，改进性能和其他修改。

- Preventive: modifications to mitigate possible degradation of usability, maintainability, etc. 预防性:通过修改来减轻可用性、可维护性等可能的退化。

Software process models

A simplified representation of a software process – an abstract representation

软件过程的简化表示——抽象表示

Depends on the system; the activities can be:

- organised in sequence 按顺序组织
- organised as interleaved 交错组织
- organised concurrently 同时组织

Must be **modelled** in order to be managed.

必须被**建模**才能被管理。

Generic models (classic)

1. The waterfall model 瀑布模型

- Separate and distinct phases 分离的和确切的阶段

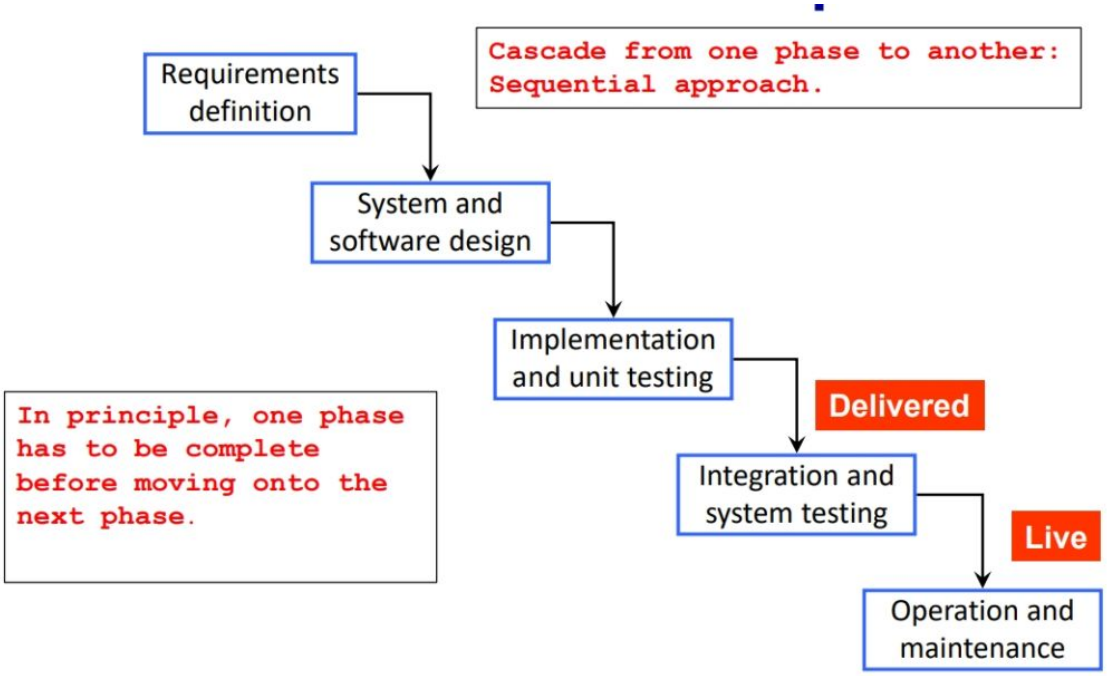
2. Evolutionary development 演化式开发

- Activities are interleaved 活动是交错的

3. RUP (The Rational Unified Process) 统一软件开发过程

- Four phases

Waterfall model



就是说，这个模型中，每一个phase都是一个一个按顺序完成的

Benefits

Easy to monitor the progress 易于监控进度

- After a small number of iterations, freeze parts of the development and continue with later phases. 在少量的迭代之后，冻结部分开发并继续后面的阶段。

Documentation is well produced at each stage.

每个阶段都有很好的文档。

Structured approach.

结构化方法

Specialised teams can be used at each stage of the lifecycle.

可以在生命周期的每个阶段使用专门的团队。

Problems

Inflexible 不可改变的

- Difficulty of accommodating change after the process is underway. 在过程进行后适应变化的困难。

Time consuming 耗费时间的

- Real projects rarely follow the sequential flow. 真正的项目很少遵循连续的流程。

- A working version of the system will not be available until late in the project time-span. 系统的工作版本直到项目时间跨度的后期才可用。

Minimises impact of global understanding over the lifecycle of a project.

最小化项目生命周期中全局理解的影响。

Not realistic.

不切合实际

Suitable projects

This model is only appropriate when the requirements are well-understood and changes will be fairly limited during the design process.

此模型仅适用于需求被充分理解，并且在设计过程中更改相当有限的情况。

- Few business systems have stable requirements!

Adaptation or enhancement of existing system.

对现有系统的调整或改进。

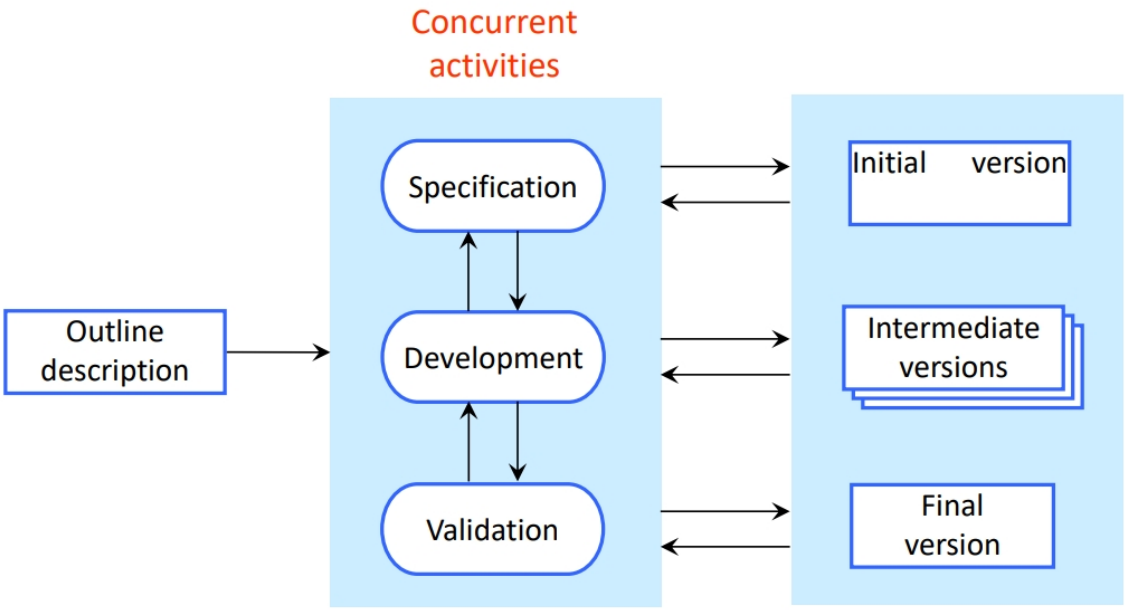
In high risk, safety critical systems e.g. air traffic control, quality is key

在高风险的安全关键系统中，如空中交通管制，质量是关键

Examples:

aircraft systems, space systems, nuclear power systems, and business critical systems, e.g. power and telecommunications

Evolutionary development



演化式开发的意思就是说，你一开始给出一个outline description，然后你和你的团队进行specification，development，validation这几个过程的循环，给出一个Initial version，再次返回这三步，继续循环出n个intermediate versions，最后得出final version

Activities are interleaved

活动是交错的

Rapid feedback

快速反馈

Refining through many versions, evolves over time

通过许多版本进行改进，随着时间的推移而发展

- Completion of a comprehensive product is impossible. 完成全面的产品是不可能的。
- Deliver core functions to meet competitive or business pressure. 交付核心功能以满足竞争或业务压力。
- Core requirements are well understood but not the detailed extension. 核心需求被很好地理解了，但却没有详细的扩展。

Benefits

Effective 有效的

- Concurrency, several members of the team may be working on different increments or releases 并发性，团队的几个成员可能在不同的增量或发布上工作

Can meet the immediate needs 可以满足突发的需要

Specification can be developed incrementally 规范可以递增地开发

Problems

Lack of process visibility 缺乏流程可见性

- Not cost-effective to produce documents that reflect every version. 生产反映每个版本的文档并不划算
- Lack of deliverable documents to measure progress. 缺乏衡量进展的可交付文件。

Systems are often poorly structured 系统通常结构不佳

- Continual change 不断改变
- Rush work 匆忙的工作

Special skills may be required 可能需要特殊技能

- E.g. in languages for rapid prototyping.

Suitable projects

Suitable for:

- small or medium-size interactive systems 中小型交互系统
- parts of large systems, e.g. the user interface 大型系统的组成部分，例如用户界面

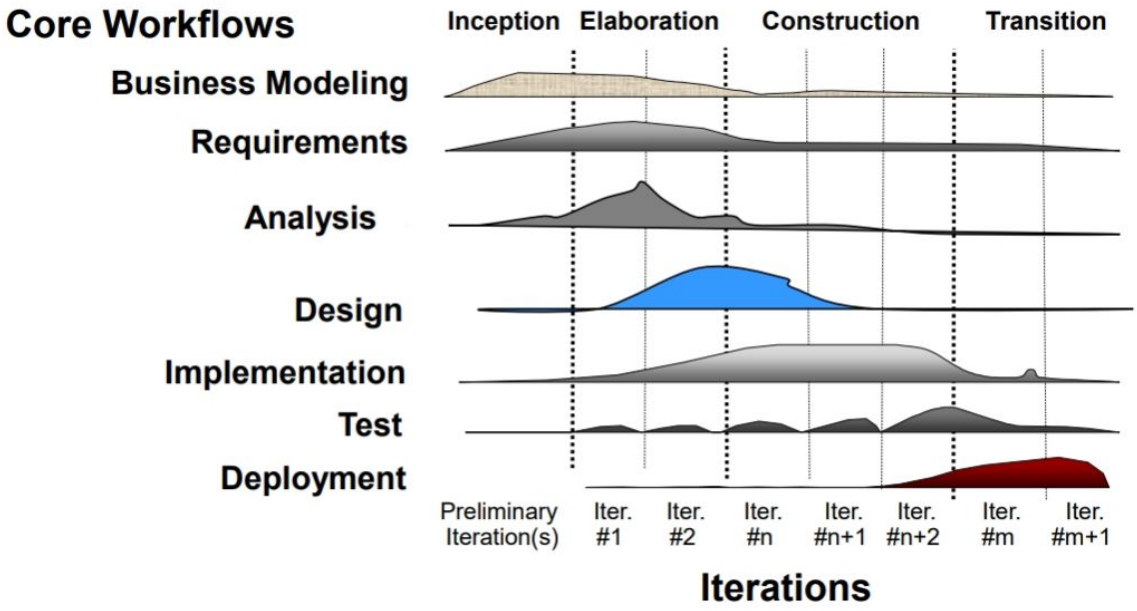
For short-lifetime systems. 对于短寿命系统。

Project with multiple features and therefore releases. 项目具有多个特性，因此会发布多个版本。

Examples:

Social networking, communication, phone apps

The Rational Unified Process (RUP)



整个上面的Inception初始，Elaboration精化，Construction构建，Transition产品化四个阶段为一个大的迭代，在每一个阶段中有小的迭代，在这些小的迭代中，每一个workflow的工作内容占比是不同的（比如初始阶段就没有design、test等）

四个步骤

Inception 初始

- ends with commitment to go ahead, business case for the project and its feasibility and feature scope identified. 以承诺继续进行，项目的业务案例及其可行性和确定的功能范围结束。

Elaboration ends with 精化结束于

- Basic architecture of the system in place. 系统的基本架构已经到位。
- A plan for construction agreed. 一份构建计划得到了同意。
- All significant risks identified. 所有重大风险已识别
- Major risks understood enough not to be too worried. 对主要风险有足够的了解，不必过于担心

Construction (iterative) 构建（迭代）

- ends with beta-release of the system. 以系统的beta版本结束。

Transition 产品化

- the process of introducing the system to its users 将系统介绍给用户的过程

Benefits

Generic process 一般性的过程

Separation of phases and workflows 阶段和工作流的分离

- Dynamic 动态的
- With goals 有目标

Problems

Overhead 开支

- Documents 文档
- Diagrams 图表

现代的开发方式统称为Agile Software Development敏捷软件开发

Problems of Traditional Development

- Poor quality 质量差
- This feature can not be tested 无法测试此特性
- Usability and User experience is bad 可用性和用户体验很差
- Can not meet the schedule 不能赶上进度
- Cost too high 费用偏高
- The team does not communicate and cooperate 团队不沟通，不合作
- Too many newcomers and lack of skills 新人太多，缺乏技能
- Too many documents 太多文档
- Is not well maintained 没有得到很好的维护

Rapid software development

The needs of Rapid Software Development:

- Rapidly changing business environments. 快速变化的商业环境。

    - No stable, consistent set of system requirements 没有稳定的、一致的系统需求集。

    - It is essential that software is developed quickly to take advantage 快速开发软件以利用这些优势是至关重要的
- Rapid development and delivery is Critical 快速开发和交付非常关键

Rapid development and delivery is now often the most important requirement for software systems 快速开发和交付现在通常是软件系统最重要的需求

- Businesses may be willing to accept lower quality software if rapid delivery of essential functionality is possible 如果能够快速交付基本功能，企业可能愿意接受质量较低的软件

The Agile Process

The processes of **specification, design, implementation and testing** are concurrent, referred to as an **iteration**:

规范、设计、实现和测试的过程是并行的，称为**迭代**:

- no detailed specification; 没有详细说明
- design documentation is minimised 设计文档被最小化

The system is developed in a series of increments 该系统是在一系列增量中开发的

- End users evaluate each increment and make proposals for later increments. 最终用户评估每个增量，并为以后的增量提出建议

End users are involved 最终用户参与其中

- System user interfaces are usually developed using an interactive development system 系统用户界面通常使用交互式开发系统开发

What is Agile?

To address the dissatisfaction with the overheads involved.

为了解决对涉及的管理费用的不满。

Agile is a set of best practices in software development based on *Scrum, Extreme Programming, Crystal Clear, DSDM, Lean* and others.

敏捷是一套基于Scrum、极限编程、Crystal Clear、DSDM、精益等的软件开发最佳实践。

The set includes:

- Iteration, TDD, continuous integration, refactoring, pair programming, story card/wall, automation test, feedback, stand up, retrospective and showcase. 迭代、TDD、持续集成、重构、结对编程、故事卡/墙、自动化测试、反馈、站立、回顾和展示。

Focus of Agile

The focus of Agile:

- Focus on the code rather than the analysis/design. 专注于代码而不是分析/设计
- Are based on an iterative approach to software development. 基于软件开发的迭代方法
- Are intended to deliver working software quickly and evolve this quickly to meet changing requirements. 旨在快速交付工作软件，并快速发展以满足不断变化的需求

The Agile Manifesto 敏捷宣言

Individuals and interactions over processes and tools

Working software over comprehensive documentation

Customer collaboration over contract negotiation

Responding to change over following a plan

个人和交互胜于过程和工具

工作软件胜过全面的文档

客户合作胜过合同谈判

对变化做出反应胜过遵循计划

Agile team 敏捷产品开发团队

Agile focuses on developers (programmers) but need other roles:

敏捷专注于开发人员(程序员)，但需要其他角色:

- business analyst, project manager, tester... 业务分析师、项目经理、测试员...

特点

Small, co-located, multi-disciplinary team, members are usually working around a table
一个小型、多学科的团队，成员通常围着一张桌子工作

- Easy communication 交流简单
- Collective code ownership 代码集体所有权
- Common vision of system ('metaphor') 系统的共同愿景("隐喻")
- Sustainable pace and common coding standard 可持续的节奏和共同的编码标准

Dos and Do NOTs in Agile

Agile needs necessary documentation

敏捷需要必要的文档

- BUT need to ensure that every document has an audience. 但是需要确保每个文档都有读者。

Agile encourages good practices

敏捷鼓励好的实践

- BUT need to ensure that every practice solves a problem. 但需要确保每一个实践都能解决一个问题。

Drop anything without value. 放弃任何没有价值的东西。

Don't over design the system. 不要过度设计系统。

Principles of Agile

Customer involvement 客户参与

- Closely involved throughout the development 密切参与整个开发过程

Incremental delivery 增量交付

- Customer specifies each increment 客户指定每个增量

People, not process 人，而不是过程

- Skills, the own way 技巧、独有方法

Embrace change 拥抱变化

- Expect change 期望变化

Maintain simplicity 维护简洁

- Both the system and the process 系统和过程

【这里给了detailed的十二条，但跟前面的也差不多】

Principles of Agile (*more detail*)

1. Our highest priority is to satisfy the customer through early and continuous delivery of valuable software.
2. Welcome changing requirements, even late in development. Agile processes harness change for the customer's competitive advantage.
3. Deliver working software frequently, from a couple of weeks to a couple of months, with a preference to the shorter timescale.
4. Business people and developers must work together daily throughout the project.
5. Build projects around motivated individuals. Give them the environment and support they need, and trust them to get the job done.
6. The most efficient and effective method of conveying information to and within a development team is face-to-face conversation.
7. Working software is the primary measure of progress.
8. Agile processes promote sustainable development. The sponsors, developers, and users should be able to maintain a constant pace indefinitely.
9. Continuous attention to technical excellence and good design enhances agility.
10. Simplicity – the art of maximizing the amount of work not done – is essential.
11. The best architectures, requirements, and designs emerge from self-organizing teams.
12. At regular intervals, the team reflects on how to become more effective, then tunes and adjusts its behaviour accordingly.

最重要的是通过尽早和不断交付有价值的软件满足客户需要。

我们欢迎需求的变化，即使在开发后期。敏捷过程能够驾驭变化，保持客户的竞争优势。

经常交付可以工作的软件，从几星期到几个月，时间尺度越短越好。

业务人员和开发者应该在整个项目过程中始终朝夕在一起工作。

围绕斗志高昂的人进行软件开发，给开发者提供适宜的环境，满足他们的需要，并相信他们能够完成任务。

在开发小组中最有效率也最有效果的信息传达方式是面对面的交谈。

可以工作的软件是进度的主要度量标准。

敏捷过程提倡可持续开发。出资人、开发人员和用户应该总是维持不变的节奏。

对卓越技术与良好设计的不断追求将有助于提高敏捷性。

简单——尽可能减少工作量的艺术至关重要。

最好的架构、需求和设计都源自自我组织的团队。

每隔一定时间，团队都要总结如何更有效率，然后相应地调整自己的行为。

【以上翻译出自百度百科——"敏捷开发"词条】

Agile的过程

核心就是这张图，接下来我们一点一点分析每一步

五个stage：Planning, Req Analysis, Design, Building, Training

Planning

Emphasis on steer, rather than precise prediction.

强调引导，而不是精确的预测。

Release planning 发布计划

- "Customer priorities" and "programmer estimates of feature difficulty" together determine release content. "客户优先级"和"程序员对功能难度的估计"共同决定了发布内容。

Iteration planning 迭代计划

- Two week delivery cycles 两周交货周期

Goal: visible progress.

目标:可见的进步。

Requirements（User stories）

User requirements are expressed as user stories.

用户需求表示为用户故事。

These are written on cards and the development team break them down into implementation tasks.

这些都写在卡片上，开发团队将它们分解为实现任务。

- These tasks are the basis of schedule and cost estimates. 这些任务是计划和成本估算的基础。

The customer chooses the stories for inclusion in the next release, based on their priorities and the schedule estimates

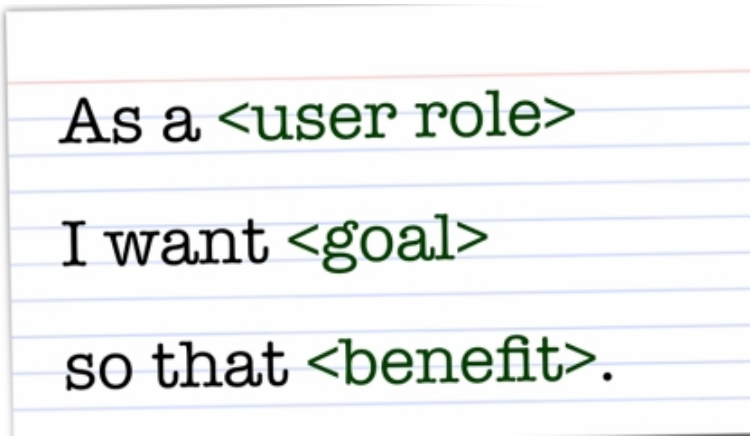
客户根据他们的优先级和时间表估算，选择要包含在下一个版本中的故事

写User Stories的格式

As a...(用户是什么角色)

I want...（用户所需要的目标）

so that...（实现目标的好处）



这个在后面的内容里我们会说的很详细

Design Improvement

Emphasis on simple design and refactoring, i.e. improving existing code.

强调简单的设计和重构，即改进现有的代码。

Removing duplication: 删除重复

- This will inevitably creep in with incremental development. 这将不可避免地在增量开发中蔓延。

Increasing cohesion 增加凝聚力

Reducing coupling 减少耦合

Extreme programming 极限编程

Perhaps the best-known and most widely used agile method.

这可能是最知名、使用最广泛的敏捷方法。

Extreme Programming (XP) takes an 'extreme' approach to iterative development:

极限编程(XP)采用了一种"极端"的迭代开发方法:

- New versions may be built several times per day. 新版本可能每天构建几次
- Increments are delivered to customers every 2 weeks 增量每两周交付给客户
- All requirements are expressed as user stories 所有需求都表示为用户故事
- Programmers work in pairs 程序员结对工作
- Develop tests before writing code 在编写代码之前开发测试
- All tests must be run for every build and the build is only accepted if tests run successfully 必须为每个构建运行所有测试，并且只有在测试成功运行时才接受构建

Pair programming

Programmers work in pairs, sitting together to develop code:

程序员结对工作，坐在一起开发代码:

- This helps develop common ownership of code and spreads knowledge across the team. 这有助于开发代码的共同所有权，并在整个团队中传播知识。
- It serves as an informal review process, as each line of code is looked at by more than 1 person. 它作为一种非正式的审查过程，因为每一行代码都由不止一个人查看
- It encourages refactoring, as the whole team can benefit from this. 它鼓励重构，因为整个团队都可以从中受益。
- Measurements suggest that development productivity with pairprogramming is similar to (or more efficient than) that of two people working independently 测量结果表明，结对编程的开发效率与两个人独立工作的开发效率相似(或更高)

Test Driven Development (TDD)

Define both an interface and a specification.

定义接口和规范。

Writing tests before code clarifies the requirements to be implemented.

在代码之前编写测试以确定要实现的需求。

Incremental test development from scenarios 来自场景的增量测试开发

- short cycles of adding tests then making them work. 短周期的添加测试，然后使它们工作

Automated test harnesses are used to run all component tests each time that a new release is built 每次构建新版本时，自动测试工具都用于运行所有组件测试

User involvement in test development and validation 用户参与测试开发和验证

- both programmer (unit) tests and customer (acceptance) tests 包括程序员(单元)测试和客户(验收)测试

Integration and release 整合和发布

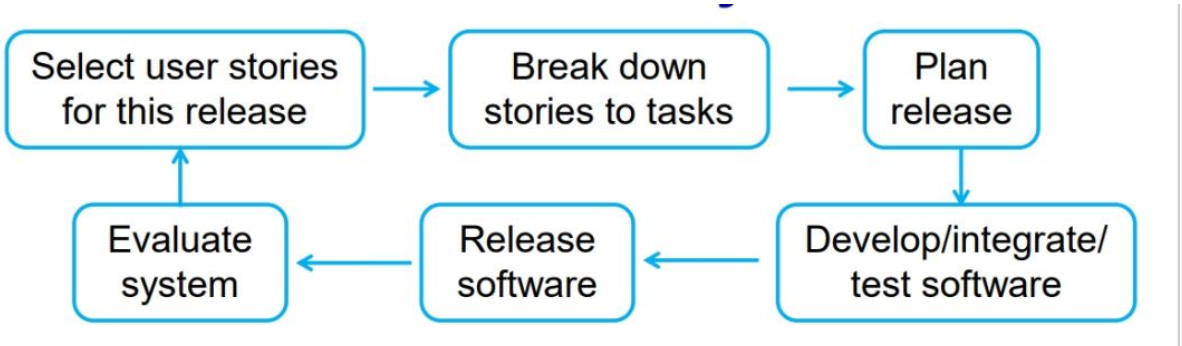
Frequent integration: 频繁的集成:

- multiple builds per day 每天构建多个版本
- everyone involved; 每个人都参与
- automated tool support. - 自动维护支持工具

Small & Frequent Releases: 小而频繁的发布:

- Team releases running, tested software delivering business value, as determined by the customer, at every iteration 团队在每次迭代中发布运行的、经过测试的软件，交付由客户确定的业务价值

The Release Cycle



Release的过程（iteration），也是一个循环，毕竟是“frequent”的release

Agile Problems

It can be difficult to keep the interest of customers who are involved in the process.

要保持参与过程的客户的兴趣是很困难的

Team members may be unsuited to the intense involvement that characterises agile methods.

团队成员可能不适合敏捷方法的高强度参与

Prioritising changes can be difficult where there are multiple stakeholders.

在有多个涉众的情况下，确定变更的优先级可能会很困难

Maintaining simplicity requires extra work.

保持简洁需要额外的工作

Contracts may be a problem as with other approaches to iterative development

与迭代开发的其他方法一样，契约可能是一个问题

Agile requires

【各种品质，随便看看就得了】

- Experience
  - Has experienced leadership
  - Hire an experienced team
- Working environment
  - Motivated stable teams
  - Supportive management
  - Team dynamics is critical
- Value thinking
- Effective and Efficient communication
- Information sharing
- Tools and Automation

Agile – suitable projects

Small or medium-sized product

中小型产品

Requirement is not clear and/or keep changing

要求不明确和/或不断变化

Rapid delivery

快速交付

A clear commitment from the customer to become involved in the development process

客户对参与开发过程的明确承诺

Not a lot of external rules and regulations that affect the software

没有太多影响软件的外部规则和条例

接下来详细说说requirement和user stories

What is a requirement?

A requirement is a feature that your system must have in order to satisfy the customer.

需求是您的系统为了满足客户而必须具备的特性

What the system should do.

系统应该做什么

It may range from a high-level abstract statement of a service or a low-level detailed specific system constraint

它的范围可以是高级的服务抽象语句，也可以是低级的详细的特定系统约束

Stakeholder 利益相关者

Any person or organization who is affected by the system in some way and so who has a legitimate interest

任何一个人或组织在某种程度上受到这个系统的影响，因此他们有合法的利益

Stakeholders view the system differently 利益相关者对系统有不同的看法

- perspective 视角
- priorities 优先处理的事

A system may not always have an identifiable stakeholder who sets the requirements. Some software is developed according to an organisation's perception of market demand

系统可能并不总是有一个可识别的利益相关者来设定需求。有些软件是根据组织对市场需求的感知开发的

【就是说stakeholders不是一定有的，requirements可能是根据市场需求自己分析出来的】

Identify the requirements（简略步骤）

- Talk to the stakeholders

- Ask questions
- Brainstorm
- Look at the **problem** from many points of view

第四步看的是什么problems?

Understanding the requirements is among the most difficult tasks that face a software engineer:

理解需求是软件工程师面临的最困难的任务之一:

- Does the customer know what is required?
- Does the end-user have a good understanding of the features and functions that will provide benefit?
- What they said = what they mean?
- Good communication?
- Even if the answers are all yes: requirements will change, and will continue to change throughout the project

这些problems的解决方法——Requirements Engineering

Requirement Engineering

【大概了解，主要记住这个名字就行】

Helps software engineers to better understand the problem they will work to solve.

帮助软件工程师更好地理解他们要解决的问题。

Software engineers and other stakeholders all participate in requirement engineering.

软件工程师和其他涉众都参与需求工程。

The extra time invested will save time and money in the long term.

从长远来看，额外投入的时间将节省时间和金钱

从ecomomic的角度来看为什么一开始确定requirement相当重要

Relative cost to fix an error:

Phase in which found	Cost ratio
requirements	1
design	3-6
coding	10
development testing	15-40
acceptance testing	30-70
operation	40-1000

... and these figures are considered conservative!

可以看到，在修改一个错误的时候，只有requirement的cost ratio(成本比率，越低说明修复该错误的耗费越少)是最低的，所以一开始确定好requirement可以给整个开发过程省大钱

Requirement classification 需求分类

分两类：

- Functional requirements 功能性需求
    - Services 服务
- Non-functional requirements 非功能性需求
    - Constraints: timing constraints, constraints on the development process, standards, etc 约束:时间约束、开发过程的约束、标准等

Functional requirements

Describe what the system should do – Functionality

描述系统应该做什么——功能

- For example in a Virtual learning environment (VLE) system

- The lecturer shall be able to set an assignment and its deadline.
- The student shall be able to submit coursework.
- The student shall be able to and view marks.
- The administrator shall be able to enter all the module information.
- The administrator shall be able to create a student list.
- The student shall receive a notification of the announcement.
- The lecturer shall be able to provide feedback comments.

【不用懂具体意思，就是什么角色有什么职责】

Functional requirements problems

Imprecision 不严密

- Problems arise when requirements are not precisely stated. 当需求没有被精确地表述时，问题就出现了。
- Ambiguous requirements may be interpreted in different ways by developers and users. 不明确的需求可能会被开发人员和用户以不同的方式解释。

Should have …

- Completeness: include descriptions of all facilities required. 完整性:包括所有所需设施的描述。
- Consistency: no conflicts or contradictions in the descriptions of the system facilities. 一致性:对系统设施的描述没有冲突或矛盾。

Non-functional requirements

Define system properties and constraints 定义系统属性和约束

- Reliability, response time, storage requirements. 可靠性，响应时间，存储要求。
- I/O device capability, system representations, etc. I/O设备能力、系统表示等

Process requirements 程序要求

- Quality standard, programming language, development method, etc. 质量标准、编程语言、开发方法等

Non-functional requirements may be more critical than functional requirements.

非功能性需求可能比功能性需求更重要

- If these are not met, the system is useless. 如果不满足这些要求，系统就毫无用处

Non-functional examples

Product requirements (product behaviour) 产品要求(产品行为)

- **E.g.** The user interface shall be implemented using HTML5.

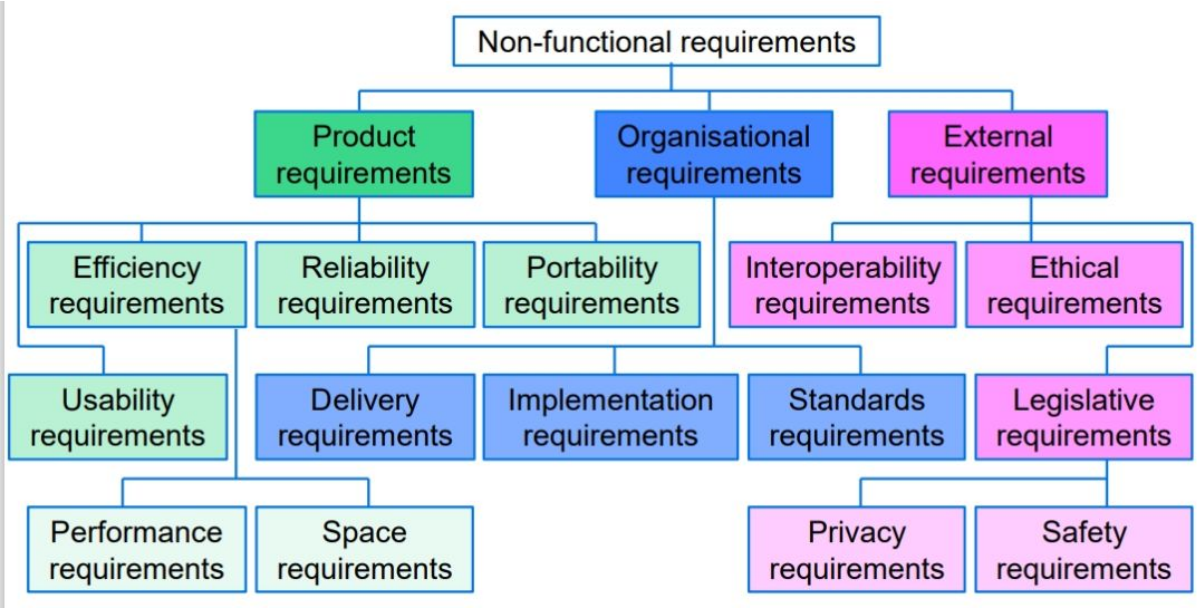
Organisational requirements (policies and procedures) 组织要求(政策和程序)

- **E.g.**The system development process shall conform to the process defined in the IT policy document.

External requirements 外部要求

- The system shall not disclose any personal information about customers, apart from their name and reference number to the operators of the system.

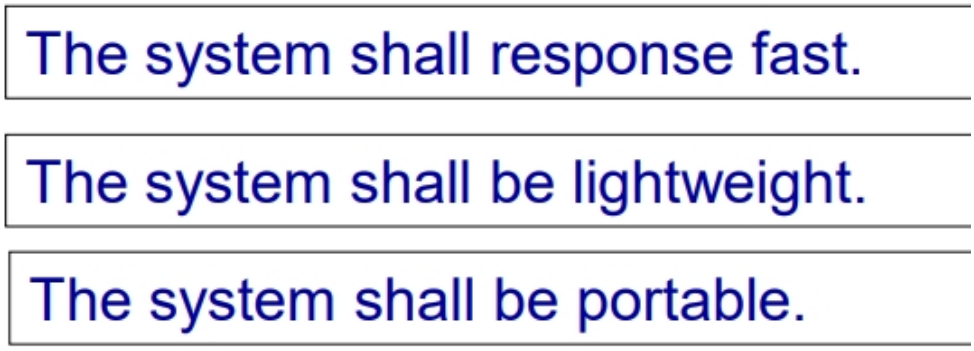
Non-functional requirements



【不算太清晰的树状图】

Non-functional requirements problems

Difficult to verify 难以核实

- 

Verifiable non-functional requirement 可验证的非功能需求

- Using some measure that can be objectively tested. 使用一些可以客观测试的测量方法。
- Quantitative 量化的

More Examples of metrics

【这里的metrics我认为是对"是否是一个好软件"的度量】

- System goal
  - The system should be easy to use by an experienced administration staff and should be organised in such a way that user errors are minimised. 该系统应易于有经验的管理人员使用，并应以用户错误最小化的方式组织。
- Verifiable non-functional requirement
  - An experienced administration staff shall be able to use all the system functions after a total of two hours training. 一个有经验的管理人员经过两个小时的培训就能使用系统的所有功能

Requirements conflicts 需求冲突

Conflicts between different requirements are common.

不同需求之间的冲突是常见的。

Example: In a digital library system

Requirement A: Item Retrieval

"This system allows the user to retrieve items in any format".

Requirement B: No File Conversion

"File conversion should not be supported".

The requirements document

Software Requirements Specification (SRS) 软件需求说明文档

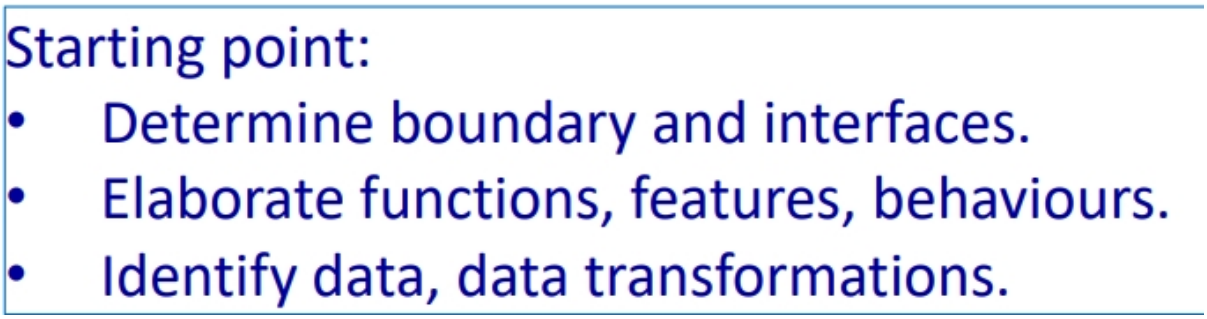
- The official statement of what is required of the system developers. 对系统开发人员的要求的官方声明
- It is NOT a design document. As far as possible, it should set out WHAT the system should do, rather than HOW it should do it. 它不是一份设计文件。它应该尽可能地说明系统应该做什么，而不是如何做。

Requirements Capture 用户需求的获取

Requirements capture:

- What is to be built? 要建造什么?
- Different starting points for each project, related to what kind of system is required, the type of organisation that requires the system, the technology etc 每个项目的起点不同，这与所需系统的类型、需要系统的组织类型、技术等有关

- e.g.



Fact-finding Techniques

一共有五种技术

- Background reading
- Interviewing
- Observation
- Document sampling
- Questionnaires

Background Reading

Learn more about the company and department the system is for, using:

了解更多关于该系统适用的公司和部门的信息，使用:

- Company reports 公司报告
- Organisation charts 组织架构图
- Policy manuals 方针手册
- Job descriptions 工作说明
- Documentation of existing system 现有系统的文档

Interviewing

【下面是一些interview的技术，感觉没啥用，记住这个是most widely used technique 就行】

Most widely used technique

- Who?

- Structured meeting, may be 1-1 or a group.
- Identify the roles that you want to interview, not the people.
- Concentrate on identifying the tasks that users complete.

Start with open-ended questions like

- “Could you tell me what you do?”
- “Why”, if the developer is an expert

Move on to more specific questions, to obtain specific information about tasks.

Direct Contact, Open Mind

Observation

Developer observes the user using the current system

开发人员观察使用当前系统的用户

Provides the developer with a better understanding of the system.

为开发人员提供对系统更好的理解。

Often during an interview, the user will concentrate on the everyday tasks and will forget about the “one-off’s”.

通常在面试过程中，面试者会把注意力集中在日常工作上，而忘记那些“一次性的”事情。

Observation refers to “planned watching of an area under study”

观察是指“有计划地观察研究区域”

Findings can be distorted if the user behaves differently.

如果用户行为不同，调查结果可能会被扭曲。

Document or Record Sampling

A specialist form of observation.

Different focuses:

- collect copies of documentation 收集文件副本
- obtain details 获取细节
- identify patterns in requirements for the system 识别系统需求中的模式

Developer should be aware that the existing forms may not be the best way to model the new system.

开发人员应该意识到现有的表单可能不是对新系统建模的最佳方式。

Ideal for capturing non-functional requirements.

非常适合捕获非功能性需求。

Questionnaires

"The aim of questionnaire design is to pose questions where misinterpretation is not possible and there is no bias".

"问卷设计的目的是在不可能产生误解和不存在偏见的情况下提出问题"。

Economical way of gathering data, ideal for the development of systems to be used by multiple users on multiple sites.

收集数据的经济方式，非常适合开发多个站点上的多个用户使用的系统。

Results could be analysed by a computer.

结果可以用计算机进行分析。

Often suffer from "questionnaire fatigue" and response is therefore low or inaccurate.

经常遭受"问卷疲劳"，因此反应低或不准确。

Difficult to write a good questionnaire

写一份好的问卷很困难

User story

In Agile process, user requirements are expressed as **user stories**.

在敏捷过程中，用户需求被表达为用户故事

- helps shift the focus from writing about requirements to talking about them. 有助于将关注点从书写需求转移到谈论需求。

These are written on cards, called **story cards**.

这些都写在卡片上，叫做故事卡。

All agile user stories include a written sentence or two and, more importantly, a series of conversations about the desired functionality

所有敏捷用户故事都包括一两句书面的句子，更重要的是，还包括一系列关于所需功能的对话

User Story的意义

The **customer** chooses the stories for inclusion in the next release based on their priorities and the schedule estimates.

客户根据他们的优先级和进度估计选择要包含在下一个版本中的故事。

The **development team** ## them down into implementation tasks. These tasks are the basis of schedule and cost estimates.

开发团队将它们分解为实现任务。这些任务是计划和成本估算的基础。

【就是客户提需求，开发团队实现需求，怎么去实现我们一会再说】

User story的确切定义（User story怎么写，格式是什么）

User stories are short, simple description of a feature

用户故事是对特性的简短描述

They typically follow a simple template:

As a *<type of user>*,
I want *<some goal>*,
so that *<some reason>*.

【主要的是前面的三行六个关键词】

Examples

As a user,
I want to backup my entire hard drive
so that I won't lose any work.

As a concerned mother,
I want to receive weather alerts for areas where my children live
so that I know when I need to worry and call them to make sure they are safe.

As a busy executive,
I want to be able to save favourites on my mobile weather application
so that I can choose from a finite drop-sown list to easily locate the weather in the destination I am travelling to.

Story cards

User stories are often written on index cards or sticky notes.

用户故事通常写在索引卡或便利贴上

As such, they strongly shift the focus from writing about features to discussing them. In fact, these discussions are more important than whatever text is written.

因此，他们强烈地将重点从编写特性转移到讨论它们。事实上，这些讨论比任何文字都重要。

Story wall

【挂story card的地方】

User stories are often stored in a shoe box, and arranged on walls or tables to facilitate planning and discussion

用户故事通常存储在鞋盒（状的容器）中，并布置在墙上或桌子上，以方便计划和讨论

Project glossary 项目术语表

It’s important to capture the language of the business domain in a project glossary.

在项目术语表中捕获业务领域的语言是很重要的。

- all terms, labels, names, etc. are clearly defined 所有的术语、标签、名称等都有明确的定义
- everyone is using the same definition 每个人都在使用相同的定义

The aim of the glossary is to define key terms and to resolve synonyms and homonyms.

术语表的目的是定义关键术语并解决同义词和同音异义词

You are building a vocabulary that you can use to discuss the system with the stakeholders.

您正在构建一个可用于与涉众讨论系统的词汇表。

【就是解释你项目里涉及的名词，来统一你小组的成员的认知】

Example:

Glossary of Terms

Term	Abbreviation	Definition	Synonym: Allowed	Synonym: Not Allowed
Administrator		This actor represents an individual with an administrative function within the LCCS. Tasks include updating and deleting customer account information, validating and rejecting user requests, blocking and unblocking customer accounts, and backing up and archiving customer and payment data.		
Blue Badge		Offers parking concessions to people with mobility difficulties or severe problems with vision. A Blue Badge holder is eligible for a 100% Congestion Charge discount.		
Cart		Provides payment of charges in the form of shopping functionality to the customer. At the technical level, the cart is represented within the system by the Order object.		
Charge Certificate		A notice issued for non-payment of the Penalty Charge.		
Charging hours		The hours between 7:00am and 6:00pm, Monday to Friday, excluding weekends, public holidays, and the period from 25 December to 1 January inclusive.		
Congestion Charge	LCC	A fee for most vehicles travelling within those parts of London designated as the Congestion Charging Zone.	London Congestion Charge	

Epics —— Large user stories

User stories can be written at varying levels of detail.

用户故事可以以不同的细节级别编写。

Large user stories are generally known as epics.

大型用户故事通常被称为史诗。

【说实话，“史诗”这个翻译很怪，但是也算是能表达出来“大型用户故事”这个意思，如果有更好的翻译可以告诉我。实在没时间去查中文的软工课程了】

Because an epic is generally too large for an agile team to complete in one iteration, it is split into multiple smaller user stories before it is worked on.

因为史诗通常太大了，敏捷团队无法在一次迭代中完成，所以在它开始工作之前，它被分割成多个较小的用户故事。

【这个分割的过程叫做"Epics break down"】

Epics break down example 1

核心就是把比较"宏大、缺少细节"的user story（Epics）来拆分成很多的细节

举个例子：

Epic

As a user,
I want to backup my entire hard drive
so that I won't lose any work.

Break down story 1

As a power user,
I want to specify files or folders to backup based on file size, date created and date modified
So that I can manage the files better.

Break down story 2

As a user,
I want to indicate folders not to backup
So that my backup drive isn't filled up with things I don't need.

- Epic只说了"我要备份整个硬盘"
- 拆成了两个stories
- 第一个对user做了细分，power user（高级用户）应该具有更高级更细节的备份权限，我认为这里是不是power user不是重点，重点是他把backup这件事情填充了细节，可以根据balabalabala这些东西来分类进行对指定文件的备份
- 第二个则是说可以选择不备份的，两个stories组合起来就是对整个backup行为的细分（我可以根据.....进行备份，也可以选择自己不备份的文件）

再看个例子：

Epic

As a busy executive,
I want to be able to save favourites on my mobile weather application
so that I can choose from a finite drop-sown list to easily locate the weather in the destination I am travelling to.

Break down story 1

As a busy executive,
I want to be able to save a search location to my list of favourites from my mobile device
So that I can reuse that search on future visits.

Break down story 2

As a busy executive,
I want to be able to name my saved searches from my mobile device
so that I know how to access each saved search in the future.

Break down story 3

As a busy executive,
I want the name of my saved searches to default to the city name, unless I choose to manually override it
So that I can streamline saving my searches.

Break down story 4

As a busy executive,
I want my “saved favourites” option on the user interface to be presented on a mobile device near the “search location”
So that I have the option of starting a new search or using a saved one, and can minimise my keystrokes within the weather application.

- busy executive 繁忙的高管
- Epic里说了要收藏某个城市的天气
- 四个Stories分别细化了这个要求，一个一个的对这个功能进行了完善和设计，更有对UI界面的设计，对于这些break down后的stories，开发人员更加明确了需求，更好的对未来的coding做准备和设计

Acceptance Criteria 验收准则

Acceptance Criteria is simply a high-level acceptance test that will be true after the agile user story is complete.

验收标准仅仅是在敏捷用户故事完成之后的高级验收测试。

Normally written on the back of the story card.

通常写在故事卡片的背面

It is a great way to ensure that a story is understood and to invite negotiation with the team about the business value that we are trying to create.

这是一种很好的方式，可以确保故事被理解，并邀请团队就我们试图创造的业务价值进行谈判

【类似我们写代码中的test环节，就是写出你这个story功能实现之后要怎么测试，也就是你的功能必须实现什么细节】

举个例子：

Story

As a vice president of marketing,
I want to select a holiday season to be used when reviewing the performance of past advertising campaigns
So that I can identify profitable ones.

Acceptance Criteria

- Make sure it works with major retail holidays: Christmas, Easter, Mother's Day, Father's Day, New Year's Day.
- Holiday seasons can be set from one holiday to the next (such as Christmas to New year's Day).
- Holiday seasons can be set to be a number of days prior to the holiday.

- 关于假期的各种细节，要求开发人员的代码必须实现这些东西

另一个例子：

Story

As a busy executive,
I want the name of my saved searches to default to the city name, unless I choose to manually override it
So that I can streamline saving my searches.

Acceptance Criteria

- Verify that the correct city name is auto populated when the search found the location by *city* and the Save option is chosen from a mobile device.
- Verify that the correct city name is auto populated when the search found the location by *postcode* and the Save option is chosen from a mobile device.
- Verify that the default location name is saved to the database.
- Verify that the default location name is populated as a saved location in future visits.
- Verify that the location name can be manfully over written if the default location name is not desired.

Non-functional Requirements as User Stories

Non-functional requirements (constraints) can also be handled as user stories.

非功能性需求(约束)也可以作为用户事例处理。

一些例子:

- As a customer, I want to be able to run your product on all versions of Windows from Windows 10 on, so that we can make use of older PCs.
- As the CTO, I want the system to use our existing orders database rather than create a new one, so that we don't have one more database to maintain.
- As a user, I want the site to be available 99% of the time I try to access it, so that I don't get frustrated and find another site to use.
- As someone who speaks Chinese, I want to have a Chinese version of the software.

【很好理解，就是这些非功能性的约束也是你整个软件里必须实现的部分，其中大部分自然也可以写成user stories这种表示需求的形式】

【当然也可以不用user stories。↓】

If you can't find a way to word the constraint, just write the constraint in whatever way feels natural

如果你找不到一种方法来表达约束，那就用任何你觉得自然的方式来写约束

Story card template 故事卡模板

【大作业用得上,QMPlus上也有】

正面（front）

Story name		Story ID
As a I want So that		
Priority	very high, high, medium, low, very low	Iteration number
Date Started		Date Finished

背面（back）

Acceptance Criteria
Notes

Who write user stories?

Anyone can write user stories.

任何人都可以编写用户故事

Over the course of a good agile project, you should expect to have user story examples written by each team member.

在一个好的敏捷项目的过程中，您应该期望每个团队成员都编写用户故事示例

Who writes a user story is far less important than who is involved in the discussions of it.

谁写用户故事远没有谁参与讨论重要

When are user stories written?

User stories are written **throughout the agile project**.

用户故事是在敏捷项目的**整个过程**中编写的。

Usually a story-writing workshop is held near the start of the agile project.

通常在敏捷项目开始的时候会举行故事编写研讨会

- Everyone on the team participates with the goal of creating a product backlog that fully describes the functionality to be added over the course of the project (or a three to sixmonth release cycle) within it. 团队中的每个人都带着创建一个产品待办事项列表的目标参与，该列表充分描述了在项目过程中(或3到6个月的发布周期)要添加的功能。

Epics will later be decomposed into smaller stories that fit more readily into a single iteration.

史诗稍后将被分解成更小的故事，更容易适合于单个迭代。

Additionally, new stories can be written and added to the product backlog at any time and by anyone.

此外，任何人都可以在任何时间编写新的故事并将其添加到产品待办事项中

Product backlog

Product backlog, which is a prioritised list of the functionality to be developed in a product or service.

产品待办事项列表，是产品或服务中要开发的功能的优先级列表。

User stories have emerged as the best and most popular form of product backlog items.

用户故事已经成为产品待办事项列表项的最佳和最受欢迎的形式。

Important: the written part of an agile user story (“As a …, I want …”) is incomplete until the discussions about that story occur.

重要的是:敏捷用户故事的书面部分(“作为……，我想……”)在关于该故事的讨论发生之前是不完整的。

It’s often best to think of the written part as a pointer to the real requirement.

通常最好是将书面部分视为指向实际需求的指针。

Product backlog (excel template)

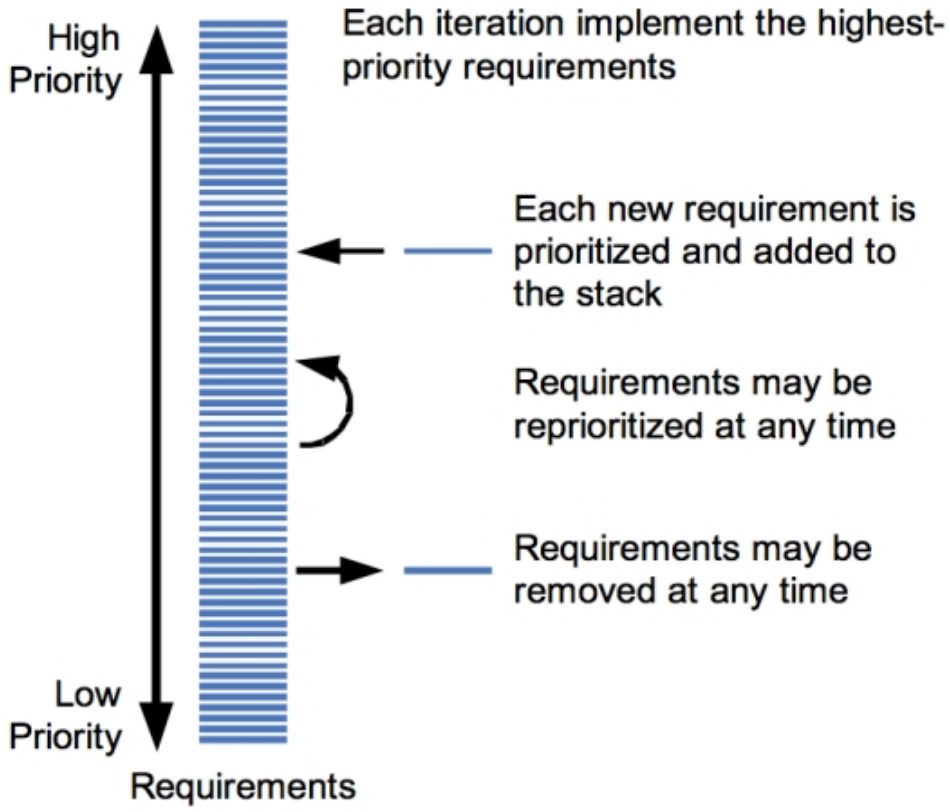
Story ID	Story Name	Description	Priority	Iteration number	Acceptance Criteria	Notes	Date started	Date finished

Prioritisation of stories 故事的优先级

Product backlog user stories must be prioritised.

必须对产品待办事项列表用户描述进行优先级排序。

- Based on business value 基于业务价值
- Value must also be supported by a positive return on investment (ROI) 价值还必须由积极的投资利润率(ROI)来支持
- Consideration of the risks 风险的考虑



【一共五个等级，Very high， high，medium，low，very low。可以在story card template里看到】

MoSCoW

A method of prioritisation favoured by the DSDM (dynamic systems development method). Elements are:

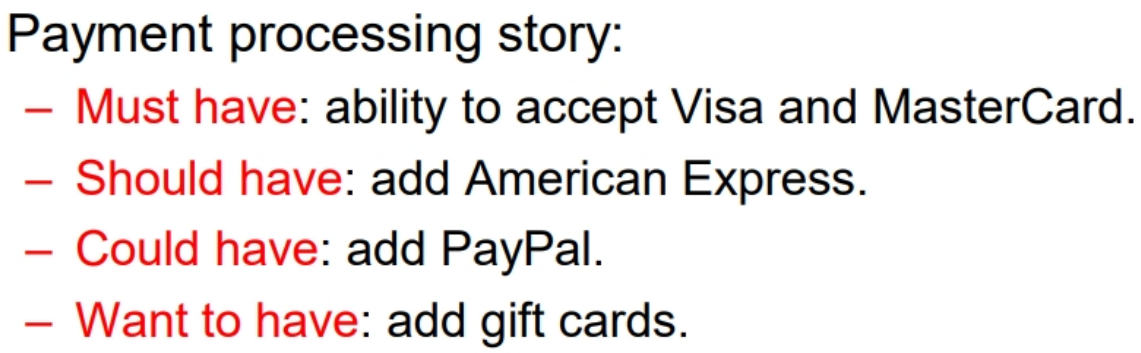
DSDM(动态系统开发方法)所青睐的优先级划分方法。元素是:

- Must have: features must be implemented, and if not, the system would not work. 必须有:必须实现特性，否则系统将无法工作。
- Should have: features are important but can be omitted if time or resources constraints appear. 应该有:特性很重要，但如果时间或资源受限，可以省略。

- Could have: features enhance the system with greater functionality, but the timelines of their delivery is not critical. 可以有:特性增强了系统的功能，但是它们的交付时间并不重要。
- Want to have: features serve only a limited group of users and do not drive the same amount of business value as the preceding items. 希望有:特性只服务于有限的用户群体，并且不能像前面的项目那样带来相同数量的业务价值。

【从上到下，越来越不重要】

举个例子：



Estimating

Estimates: how long the work is likely to take

估计:工作可能需要多长时间

Some methods include: 一些方法包括:

- Level of Effort or T-shirt sizing: small, medium, large, extra large… 努力程度或t恤尺码:小号、中号、大号、特大号……
- Ideal Time: under ideal circumstance 理想时间:在理想情况下
- Hours: actual clock hours 小时:实际时钟小时数
- Story points: an arbitrary measure that allows the teams to understand the size of the effort 故事点:允许团队了解工作规模的任意度量

Estimating story points

Story points:

- Fibonacci Sequence: 1,2,3,5,8,13,21…【别问为什么，就这么记这么用就行】

- As the points get higher, the degree of uncertainty is increasing. 随着点的升高，不确定性的程度也在增加。
- Abstract and negotiable 抽象且可协商
- Everyone on the team agrees to the size of a story point, using relative sizing 团队中的每个人都同意使用相对大小来确定故事点的大小
    - E.g. adding a field to the database, agreed as a 5 story points 向数据库中添加一个字段，约定为5故事点

如何判定Good user story? —— “INVEST”

INVEST

- Independent 独立的
- Negotiable 可协商的
- Valuable 有价值的
- Estimatable 可估计的
- Small 小的
- Testable 可测试的

Prototyping

Physical user-interface design 物理用户界面设计

- Draw the user interfaces on a paper 在纸上画出用户界面
    - Get quick feedback

Logical user-interface design (information) 逻辑用户界面设计(信息)

- Which user-interface elements are needed?
- How are these elements related to each other?
- What should they look like?
- How should they be manipulated?

Low-fidelity Prototyping 低保真原型

Use paper and sketch 使用纸张和草图

优点：

- Quick
    - No technical learning curve 没有技术学习曲线
    - Relevant feedback on early concepts 早期概念的相关反馈
    - Provides conceptual direction 提供概念指导
- Effective
    - Low investment of time/resources 时间/资源投入少
    - Fail early, fail fast 早失败，快失败
    - Expedites detailed design 加快详细设计

Medium-fidelity Prototyping 中保真原型

Limited functionality but clickable areas which presents the interactions and navigation of an application.

功能有限，但可点击的区域，呈现应用程序的交互和导航。

Usually built upon storyboards or user scenarios.

通常建立在故事板或用户场景之上。

Tools:

- Fliplet.com
- Proto.io
- Adobe XD
- And many more

High-fidelity Prototyping 高保真原型

Computer-based interactive representation of the product in its closest resemblance to the final design in terms of details and functionality.

在细节和功能方面，以计算机为基础的产品交互表现形式与最终设计最接近。

Consider user interface (UI) and the user experience (UX).

考虑用户界面(UI)和用户体验(UX)。

就是说谁越接近最后的成果，谁的原型的fidelity（保真度）越高

很形象的例子

